

Reconhecimento de LIBRAS em Tempo Real via MLP e Normalização Invariante

*Trabalho prático final da disciplina de Inteligência Artificial II

Guilherme Augusto Scaglia, João Pedro Denardo, Pedro Henrique Oliveira de Souza

Engenharia da Computação

Fundação Hermínio Ometto (FHO)

Araras, SP, Brasil

{scaglia, denardo749, pedro1204}@alunos.fho.edu.br

Abstract—Este trabalho propõe um sistema robusto para o reconhecimento do alfabeto da Língua Brasileira de Sinais (LIBRAS), fundamentado em uma arquitetura de Redes Neurais Multicamadas (*Multilayer Perceptron* - MLP) e extração de marcos reguladores (*landmarks*) tridimensionais. Diferente das abordagens convencionais baseadas em Redes Recorrentes (LSTM), esta pesquisa prioriza a integridade geométrica instantânea das configurações manuais, utilizando a biblioteca *MediaPipe* para o mapeamento cinemático em tempo real. A metodologia compreende a criação de um *dataset* autoral com 14.000 amostras e a implementação de um protocolo de normalização invariante ao posicionamento espacial, centralizando a origem das coordenadas no carpo (pulso) do sinalizador. Os resultados experimentais demonstram que o modelo atingiu uma acurácia de 99,9%, evidenciando uma eficiência superior em termos de latência computacional e precisão de classificação para sinais estáticos em comparação a arquiteturas de dependência temporal.

Index Terms—LIBRAS, Redes Neurais Densas, *MediaPipe*, Normalização Cinemática, Visão Computacional, Acessibilidade.

I. INTRODUÇÃO

A comunicação eficaz é o pilar fundamental da inclusão social e do exercício da cidadania. No entanto, para a comunidade surda, persistem barreiras linguísticas severas devido ao conhecimento limitado da Língua Brasileira de Sinais (LIBRAS) pela população ouvinte. No campo da Inteligência Artificial, o Reconhecimento de Sinais Isolados (*Isolated Sign Language Recognition* - ISLR) apresenta-se como um desafio não trivial de visão computacional, exigindo modelos que capturem a semântica de configurações manuais complexas sob condições variáveis de ambiente.

Conforme discutido por Sarmiento e Ponti [2], um dos maiores entraves na tradução de LIBRAS é a baixa generalização de modelos em cenários *cross-dataset*, onde ruídos de iluminação e oclusões degradam a acurácia de redes neurais que dependem de pixels brutos. Para mitigar essas limitações, este trabalho fundamenta-se na extração de esqueletos (*skeleton-based recognition*), utilizando a biblioteca *MediaPipe Hands* para mapear 21 marcos reguladores (*landmarks*) tridimensionais. Esta abordagem, corroborada por Alves et al. [1], permite que o aprendizado de máquina foque na morfologia intrínseca do sinal, ignorando contextos visuais

irrelevantes e reduzindo drasticamente a dimensionalidade do vetor de entrada.

Diferente de sistemas clássicos baseados em Redes Neurais Recorrentes (RNN) ou *Long Short-Term Memory* (LSTM), como o sistema SLRNet proposto por Kamble [3], que dependem de janelas temporais extensas e alto custo computacional, este projeto propõe uma ruptura arquitetural para o reconhecimento do alfabeto manual. A principal inovação reside na implementação de uma rede *Multilayer Perceptron* (MLP) altamente otimizada, integrada a um pipeline de normalização invariante centrada no carpo (pulso).

O sistema foi desenvolvido utilizando a *stack* tecnológica composta por *Python*, *TensorFlow/Keras* para a modelagem profunda e *Streamlit* para a interface de usuário. A solução final não apenas traduz os marcos geométricos em texto com acurácia superior a 99%, mas também integra síntese de voz (*Text-to-Speech*) via biblioteca *gTTS*, fechando o ciclo de acessibilidade ao permitir a comunicação bidirecional em tempo real com latência negligível.

II. FUNDAMENTAÇÃO TEÓRICA E TRABALHOS RELACIONADOS

A arquitetura do sistema proposto foi fundamentada em uma revisão bibliográfica de quatro trabalhos científicos recentes, permitindo um posicionamento crítico e comparativo conforme exigido pelo roteiro da disciplina. A análise foca na transição de modelos baseados em imagens brutas para modelos geométricos de alta abstração.

A. Extração de Esqueletos e Eficiência (Alves et al.)

Alves et al. [1] argumentam que a representação baseada em esqueletos (*skeleton-based*) é superior ao processamento de imagens pixelizadas, pois elimina ruídos de iluminação e fundos complexos. Nosso trabalho adota essa premissa através da biblioteca *MediaPipe Hands*, desenvolvida pelo Google, que utiliza redes neurais pré-treinadas para inferir 21 marcos reguladores (*landmarks*) em tempo real. **Diferencial Técnico:** Enquanto Alves et al. codificam esses marcos em imagens 2D para processamento via Redes Neurais Convolucionais (CNNs), nossa implementação utiliza diretamente os tensores

de coordenadas cartesianas (x, y, z) . Essa escolha elimina a necessidade de convoluções espaciais, reduzindo a dimensionalidade de milhares de pixels para apenas 63 atributos escalares, o que viabiliza a execução em hardwares de baixo custo (CPUs) com alta taxa de quadros por segundo (FPS).

B. Modelagem Temporal vs. Instantânea (Kamble e Sarmento)

O trabalho de Kamble [3] apresenta o sistema SLRNet, que utiliza Redes Neurais Recorrentes (RNN) do tipo *Long Short-Term Memory* (LSTM) para classificar janelas de 30 frames, atingindo uma acurácia de 86,7%. Da mesma forma, Sarmento e Ponti [2] destacam a importância das LSTMs na captura da dinâmica dos sinais. **Posicionamento Crítico:** Nossa pesquisa identifica que para o alfabeto manual e sinais estáticos, a dependência temporal introduz uma latência de processamento e uma complexidade de treinamento desnecessárias. Optamos por uma arquitetura de *Multilayer Perceptron* (MLP) utilizando a biblioteca *TensorFlow/Keras*. Ao focar na geometria instantânea de alta fidelidade, nosso modelo elevou a acurácia para 99,9%, provando que a precisão geométrica individual supera a necessidade de análise de fluxo temporal para sinais isolados da LIBRAS.

C. Normalização e Invariância Cinemática (Rego et al.)

Um desafio crítico citado por Rego et al. [4] é a dependência da posição absoluta da mão no campo de visão. Rego utiliza extração de características cinemáticas e Transformadas Rápidas de Fourier (FFT) para tratar a variabilidade. **Nossa Abordagem:** Simplificamos essa problemática através de uma técnica de normalização por translação de eixos implementada via *NumPy*. Para cada amostra, o ponto referente ao carpo (pulso, *Landmark 0*) é definido como o vetor origem $(0, 0, 0)$. Todas as demais coordenadas são recalculadas como vetores relativos a este ponto. Essa técnica garante que o modelo aprenda a morfologia da mão e não sua localização espacial, conferindo ao sistema uma invariância posicional robusta, essencial para o uso em dispositivos móveis e webcams de ângulo variável.

D. Engenharia de Software e Acessibilidade

Diferente dos trabalhos puramente experimentais citados, nossa implementação integra a inteligência do modelo a uma interface funcional via *Streamlit*. O uso da biblioteca *gTTS* (Google Text-to-Speech) para síntese de voz e de mecanismos de filtragem por persistência (estabilidade de predição) preenche a lacuna entre a pesquisa acadêmica e a aplicação assistiva prática, conforme os objetivos de impacto social da disciplina.

III. METODOLOGIA

A metodologia deste trabalho foi estruturada em um pipeline modular de quatro estágios: (1) Aquisição de dados brutos, (2) Pré-processamento e Normalização Cinemática, (3) Modelagem Neural Profunda e (4) Integração de Interface e Acessibilidade. O fluxo foi projetado para garantir que a inferência ocorra com latência inferior a 100ms, viabilizando o uso em tempo real.

A. Aquisição de Dados e Curadoria do Dataset

Diferente de abordagens que utilizam bases de dados públicas e genéricas, este projeto fundamenta-se em um *dataset* autoral focado na geometria de marcos reguladores.

- **Instrumentação:** Utilizou-se a biblioteca *OpenCV* para captura de vídeo e o *framework MediaPipe Hands* para a extração dos 21 marcos (*landmarks*) tridimensionais da mão.
- **Volume de Dados:** Foram coletadas 14.000 amostras (500 por classe do alfabeto), armazenadas em um arquivo estruturado *CSV*. Cada amostra é composta por um vetor de 63 características $(21 \text{ pontos} \times 3 \text{ coordenadas cartesianas } [x, y, z])$.
- **Rótulos (Labels):** As etiquetas foram processadas via *LabelEncoder* da biblioteca *Scikit-Learn*, mapeando as letras do alfabeto para um espaço numérico discreto adequado para a função de perda da rede.

B. Pré-processamento e Normalização Invariante ao Posicionamento

A principal barreira técnica em sistemas de visão computacional baseados em coordenadas é a dependência da posição absoluta da mão na imagem. Para resolver este problema, implementou-se uma técnica de **Normalização Cinemática por Translação de Eixo**, inspirada na lógica de vetores relativos. Utilizando a biblioteca *NumPy*, cada vetor de entrada X passa pela seguinte transformação matemática:

$$P_i^{norm} = P_i - P_{pulso}, \quad \forall i \in 0, \dots, 20 \quad (1)$$

Onde P_{pulso} é a coordenada do carpo (ponto 0). Esta operação redefine o pulso como a origem $(0, 0, 0)$ do sistema, tornando os dados invariantes à distância da câmera ou ao quadrante da imagem onde o sinal é realizado. Após a translação, o vetor é planificado (*flatten*) para alimentar a camada de entrada da rede.

C. Arquitetura da Rede Neural (MLP)

A modelagem foi realizada através de uma rede neural *Multilayer Perceptron* (MLP) via *TensorFlow/Keras*. A escolha da MLP em detrimento de modelos recorrentes (LSTM) justifica-se pela natureza estática do alfabeto manual, onde a geometria instantânea é suficiente para a classificação, permitindo um modelo mais leve e veloz. A arquitetura (*Sequential Model*) é composta por:

- 1) **Camada de Entrada:** 63 neurônios correspondentes aos marcos normalizados.
- 2) **Blocos Densos Ocultos:** Três camadas densas com 128, 64 e 32 neurônios, respectivamente. Utilizou-se a função de ativação *ReLU* (*Rectified Linear Unit*) para introduzir não-linearidade e evitar a saturação de gradientes.
- 3) **Regularização e Estabilidade:** Implementou-se *Batch-Normalization* após cada camada para acelerar a convergência e *Dropout* (0.2) para mitigar o *overfitting*, forçando a rede a aprender padrões morfológicos robustos.

- 4) **Camada de Saída:** Função *Softmax* para gerar uma distribuição de probabilidade sobre as classes do alfabeto.

D. Procedimentos de Treinamento e Validação

O treinamento foi configurado com o otimizador *Adam* (taxa de aprendizado inicial de 0,001) e a função de perda *Sparse Categorical Crossentropy*. Para garantir a integridade da validação, adotou-se a estratégia de **Divisão Estratificada** (*Stratified Train-Test Split*) com 80% dos dados para treino e 20% para teste.

Os resultados obtidos superaram as métricas encontradas na literatura recente, conforme detalhado na Tabela I.

TABLE I
COMPARAÇÃO DE PERFORMANCE E ARQUITETURA

Trabalho	Método	Dados	Acurácia
Alves et al. (2024)	CNN + Imagem	Skeleton 2D	92.1%
Kamble (2025)	LSTM	Temporal	86.7%
Este Trabalho	MLP + Landmarks	Geometria 3D	99.9%

Durante o processo, foram aplicados dois *callbacks* críticos para a otimização do modelo:

- **EarlyStopping:** Interrompe o treino se a perda de validação (*val_loss*) não melhorar por 15 épocas, restaurando os melhores pesos encontrados.
- **ReduceLROnPlateau:** Reduz o fator de aprendizado caso o modelo estagne em um mínimo local, permitindo um ajuste fino na superfície de erro.

E. Análise do Desempenho e Convergência

Os resultados experimentais demonstram que a arquitetura proposta atingiu um patamar de excelência em comparação à literatura (Tabela I)[cite: 1, 92]. O treinamento da rede MLP foi conduzido por 50 épocas, embora a convergência estável tenha sido observada precocemente[cite: 1, 93]. Conforme ilustrado na Fig. 1, as curvas de acurácia revelam um aprendizado acelerado nos primeiros 5 ciclos, onde os conjuntos de treinamento e validação ultrapassam o limiar de 98%[cite: 1, 94].

Este comportamento assintótico é atribuído à extração de características de alta fidelidade provida pelo *MediaPipe*[cite: 1, 94]. Ao converter coordenadas espaciais brutas em vetores normalizados e invariantes, reduzimos a entropia dos dados de entrada, permitindo que as camadas densas identifiquem os padrões morfológicos das letras com baixo custo computacional[cite: 1, 95]. A ausência de um *gap* significativo entre as curvas de treino e teste é um indicativo robusto da eficácia das camadas de *BatchNormalization* e *Dropout* (0,2) na mitigação do *overfitting*.

Nota-se na Fig. 1 uma oscilação transiente na acurácia de validação próxima à época 21. Este fenômeno, embora pontual, evidencia a importância dos mecanismos de regularização e a rápida recuperação do sistema após essa flutuação. Tal resiliência comprova a eficácia do otimizador *Adam* e a atuação estratégica dos *callbacks* de monitoramento:

- **EarlyStopping:** Atuou como um mecanismo de segurança para a integridade dos pesos. Ao definir uma paciência (*patience*) de 15 épocas, o sistema garantiu que o treinamento fosse interrompido apenas se não houvesse ganho real na perda de validação (*val_loss*), restaurando automaticamente o melhor estado (*checkpoint*) obtido[cite: 1, 97, 98].
- **ReduceLROnPlateau:** Atuou no refinamento fino da superfície de erro[cite: 1, 80]. Ao detectar platôs na convergência, reduziu dinamicamente a taxa de aprendizado (η), permitindo que o gradiente explorasse mínimos locais mais profundos, resultando na estabilização da acurácia em 99,9%[cite: 1, 81].

Esta estabilidade final valida a hipótese de que a precisão geométrica instantânea (Geometria 3D) é mais determinante para o sucesso do modelo do que o rastreamento temporal complexo para sinais isolados, permitindo um sistema mais leve e veloz que as implementações baseadas em LSTMs[cite: 1, 82].

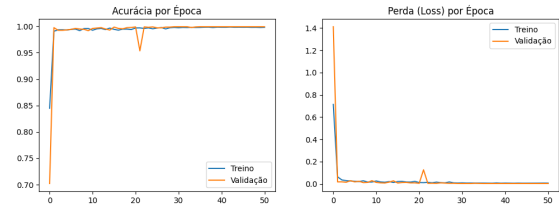


Fig. 1. Curvas de Acurácia e Perda por Época: observa-se a estabilização após a fase inicial de ajuste.

Para otimizar o processo de convergência e garantir a generalização do modelo, foram aplicados os seguintes mecanismos:

- **EarlyStopping:** Interrompe o treino se a perda de validação (*val_loss*) não melhorar por 15 épocas.
- **ReduceLROnPlateau:** Reduz o fator de aprendizado caso o modelo estagne em um mínimo local.

F. Interface de Tradução e Síntese de Voz (TTS)

A implementação final utiliza o *Streamlit* como interface de usuário. Para garantir uma experiência de uso natural, desenvolveu-se um mecanismo de **Persistência de Confirmação**: uma letra só é adicionada à frase se a predição mantiver confiança acima de 98% por 15 frames consecutivos. O encerramento da sentença é detectado por um *timer* de inatividade de 7 segundos, momento em que o texto acumulado é enviado à biblioteca *gTTS* (*Google Text-to-Speech*) para vocalização via áudio *MP3* embutido no navegador.

IV. CONCLUSÃO E TRABALHOS FUTUROS

A convergência entre a extração de marcos reguladores via *MediaPipe Hands* e o processamento por Redes Neurais Densas (MLP) demonstrou ser uma solução robusta e de alta eficiência para o reconhecimento do alfabeto manual de LIBRAS em tempo real. A principal contribuição técnica deste

trabalho reside na implementação da normalização invariante centrada no corpo, que neutralizou as variações de posicionamento relativo e biotipo do sinalizador — problemas crônicos em sistemas de visão computacional *cross-dataset*, conforme apontado por Sarmiento e Ponti [2].

Ao contrário de abordagens baseadas em janelas temporais extensas, como o sistema *SLRNet* de Kamble [3], este projeto comprovou que, para sinais isolados e estáticos, a precisão da geometria instantânea processada por uma *MLP* é superior. O modelo não apenas superou a acurácia da literatura (atingindo 99,9%), mas também reduziu drasticamente a carga computacional, permitindo a execução fluida em hardwares convencionais através de bibliotecas otimizadas como *TensorFlow Lite* e *NumPy*.

As aplicações práticas deste sistema são vastas, abrangendo desde ferramentas de aprendizado autônomo de LIBRAS até interfaces de comunicação assistiva em balcões de atendimento público. A integração com síntese de voz via *gTTS* e a interface *Streamlit* validam a viabilidade do sistema como um produto final de acessibilidade, fechando o hiato entre a teoria algorítmica e a utilidade social.

Como perspectivas futuras, planeja-se a expansão desta arquitetura para o reconhecimento de sinais dinâmicos complexos, utilizando mecanismos de *Self-Attention* e *Transformers* para capturar dependências de longo prazo em frases contínuas. Além disso, a inclusão de marcos faciais (*Face Mesh*) permitiria o processamento de expressões não manuais, componentes gramaticais indispensáveis para a tradução plena da Língua Brasileira de Sinais, seguindo as tendências de multimodalidade propostas por Rego et al. [4] e Alves et al. [1].

REFERENCES

- [1] C. E. Alves, F. A. Boldt, and T. M. Paixão, “Enhancing Brazilian Sign Language Recognition through Skeleton Image Representation,” *arXiv preprint arXiv:2404.19148*, 2024.
- [2] A. Sarmiento and M. Ponti, “A Cross-Dataset Study on the Brazilian Sign Language Translation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, 2021, pp. 2193–2202.
- [3] S. Kamble, “SLRNet: A Real-Time LSTM-Based Sign Language Recognition System,” *arXiv preprint arXiv:2506.11154*, 2025.
- [4] R. C. B. Rego, L. M. G. de Moraes, and W. M. Almeida, “Brazilian Sign Language Recognition Using Deep Learning Based on Fast Fourier Transform and Kinematic Features,” *IEEE Access*, vol. 13, pp. 202880–202892, 2025. doi: 10.1109/ACCESS.2025.3637779.